

# COMPLETE FLOORPLANNING Physical Design Interview Preparation

---

NVIDIA · Intel · STMicroelectronics · Google · Qualcomm · AMD · Apple · Broadcom ·  
Samsung · TSMC · Cadence · Synopsys

19 Topics · 4 Interview Questions Each · 76 Total Q&A

---

**Questions in RED | Answers in BLUE**

# TABLE OF CONTENTS

---

1. Netlist (Input to Floorplanning)
2. Calculate Core Area
3. Calculate Die Area
4. Set Utilization
5. Aspect Ratio (AR)
6. Create Core Boundary
7. Place IO Pads
8. Place Macros
9. Macro Clustering
10. Flyline Analysis
11. Macro Optimization
12. Channel Spacing
13. Halo Creation
14. Routing Channel Creation
15. Placement Blockages
16. Check Congestion
17. Check Macro Overlaps
18. Floorplan Verification
19. Power Planning

## 1. Netlist (Input to Floorplanning)

---

### Q1. What is a netlist and what information does it carry into the floorplanning stage? (Intel, STMicroelectronics)

*Asked at: Intel, STMicroelectronics, Synopsys*

A netlist is a structural description of a circuit listing all logical cells (gates, flip-flops, macros) and their interconnections (nets). It carries: cell types and names, net connectivity between cells, hierarchical module information, timing constraints (via SDC), port definitions, and technology library references. The post-synthesis netlist fed into floorplanning includes estimated cell counts, hard macro instances (SRAMs, IPs), and connectivity data critical for estimating core area, utilization, and macro placement decisions during floorplanning.

---

### Q2. What is the difference between a gate-level netlist and an RTL netlist? (NVIDIA, Qualcomm)

*Asked at: NVIDIA, Qualcomm, AMD*

An RTL (Register Transfer Level) netlist describes circuit behavior using hardware description language (Verilog/VHDL) with always blocks and assign statements — technology-independent and synthesizable. A gate-level netlist is the output of logic synthesis, mapping RTL to actual standard cells from a specific technology library. Floorplanning always uses a gate-level netlist because it contains actual cell footprints, drive strengths, and macro instances needed to estimate area and congestion accurately. RTL is too abstract to provide physical area or timing information.

---

### Q3. What checks are performed on the netlist before starting floorplanning? (Google, Broadcom)

*Asked at: Google, Broadcom, Cadence*

Pre-floorplanning netlist checks include: Design Rule Check (DRC) on the netlist (unconnected ports, floating nets, multiple drivers), SDC constraint validation (clock definitions complete, all I/O constrained), library reference checks (all cells exist in the technology library), logical DRC (combinational loops, unresolved black boxes), area estimation sanity check, macro instantiation verification (all hard macros instantiated correctly), scan chain verification (DFT netlist integrity), and power domain checks (UPF/CPF consistency with netlist hierarchy). A clean netlist is mandatory before investing time in physical floorplanning.

---

### Q4. How does netlist hierarchy affect floorplanning decisions? (Apple, AMD)

*Asked at: Apple, AMD, Samsung*

Netlist hierarchy defines logical partitions (blocks, sub-blocks) that often correspond to physical partitions in the floorplan. Hierarchical designs are physically partitioned into voltage areas, power domains, or rectilinear regions that match the logical hierarchy for easier timing closure. High connectivity between two hierarchical blocks suggests they should be placed adjacent in the floorplan to minimize inter-block wirelength. Hierarchy also enables hierarchical physical design flows where blocks are independently implemented and assembled, requiring well-defined interface timing budgets at block boundaries.

---

## 2. Calculate Core Area

---

### Q1. How is core area calculated during floorplanning? (Intel, NVIDIA)

*Asked at: Intel, NVIDIA, TSMC*

Core area is calculated as:  $\text{Core Area} = \text{Total Cell Area} / \text{Target Utilization}$ . Total cell area includes standard cell area (from synthesis reports), hard macro area (SRAMs, IPs, analog blocks), and estimated area for buffers/fillers added during implementation. If target utilization is 70%, and total cell area is 10 mm<sup>2</sup>, then core area =  $10 / 0.70 = 14.28 \text{ mm}^2$ . The remaining 30% (4.28 mm<sup>2</sup>) is white space for routing, clock trees, filler cells, and placement flexibility. Utilization targets typically range from 60-85% depending on design complexity and routing demand.

---

### Q2. What factors determine the target utilization chosen for core area calculation? (STMicroelectronics, Qualcomm)

*Asked at: STMicroelectronics, Qualcomm, Cadence*

Target utilization depends on: routing complexity (more metal layers allow higher utilization), process node (advanced nodes have tighter routing rules, requiring lower utilization), clock tree area overhead (large clock trees consume significant white space), number of power domains (domain boundaries reduce usable area), macro density (macros displace standard cells, requiring surrounding white space), design criticality (timing-critical designs need lower utilization for optimization flexibility), and historical data from similar designs. Typical ranges: 65-70% for complex SoCs, 75-80% for simpler blocks, rarely exceeding 85%.

---

### Q3. What is the difference between core area and die area? (AMD, Google)

*Asked at: AMD, Google, Apple*

Core area is the inner rectangular region where all standard cells, macros, and routing exist — the active logic area. Die area is the total chip area including the core area plus the IO ring (pad frame) containing IO cells, ESD structures, power/ground pads, and corner cells.  $\text{Die area} = \text{Core Area} + \text{IO Ring Area}$ . The IO ring adds significant area, especially for pad-limited designs where the die size is determined by the number of IO pads rather than logic area. For core-limited designs, the core area directly determines die size.

---

### Q4. How do hard macros affect core area estimation? (Broadcom, Samsung)

*Asked at: Broadcom, Samsung, Synopsys*

Hard macros (SRAMs, ROM, analog IPs, PLLs) have fixed footprints specified by their datasheets or characterization files. Their area is added directly to total cell area before applying utilization. However, macros also create placement halos (keep-out zones) around them where standard cells cannot be placed, effectively increasing the area impact beyond their physical footprint. Additionally, macros require routing channels between them and around their boundaries. These overhead areas must be accounted for in core area estimation, often requiring 10-20% additional area beyond naive macro area summation.

---

## 3. Calculate Die Area

---

### Q1. How is die area calculated and what are the two limiting factors? (Intel, STMicroelectronics)

*Asked at: Intel, STMicroelectronics, TSMC*

Die area is calculated considering two potential limiting factors: (1) Core-limited: Die Area = Core Area + IO ring overhead. Core area drives die size when logic density is high. (2) IO-limited (pad-limited): Die size is determined by the number of IO pads needed multiplied by pad pitch, because the IO ring perimeter must accommodate all pads. Die Area = (Total IO pads \* pad pitch / 4)<sup>2</sup> for a square die. The actual die uses whichever constraint gives the larger area. Many modern designs are core-limited, with IO pads staggered or using flip-chip bumps.

---

### Q2. What is the aspect ratio and how does it influence die area shape? (NVIDIA, Qualcomm)

*Asked at: NVIDIA, Qualcomm, AMD*

Aspect ratio (AR) is the ratio of die height to die width (AR = Height/Width). AR = 1.0 gives a square die; AR > 1 gives a taller die; AR < 1 gives a wider die. Aspect ratio is chosen based on: IO pad distribution (balanced pads on all sides favor square dies), package constraints (package shape may dictate AR), macro shapes and natural floorplan arrangement, and manufacturing preferences (square dies are often preferred for wafer packing efficiency). AR between 0.5 and 2.0 is typical. Extreme aspect ratios create long wire paths and routing difficulty.

---

### Q3. How does the 1:1 (square) vs 2:1 (rectangle) die shape impact physical design? (Google, Apple)

*Asked at: Google, Apple, Broadcom*

A square die (1:1 AR) typically offers: balanced routing in both X and Y directions, equal IO pad distribution on all four sides, and simpler floorplanning. A 2:1 rectangular die has: longer dimension that may cause clock skew challenges across the long axis, potential routing congestion in one direction, but may better fit package constraints or specific macro arrangements. Rectangular dies are sometimes used when a dominant functional block (wide datapath, long memory array) naturally fits a rectangular form. The choice significantly impacts power grid design, clock tree topology, and overall routing quality.

---

### Q4. What package constraints influence die area and shape decisions? (AMD, Marvell)

*Asked at: AMD, Marvell, MediaTek*

Package constraints include: package body dimensions (die must fit within package with adequate bond wire length or bump pitch), pin count and assignment (ball grid array pitch determines minimum die size for flip-chip), thermal requirements (larger die area may be needed for heat spreading), package type (wire bond packages prefer smaller dies with peripheral IO; flip-chip allows distributed IO bumps across the entire die), ESD and reliability constraints (minimum pad-to-pad spacing), and cost (larger die means fewer dies per wafer, increasing cost). Package and die co-design is essential for optimal results.

---

## 4. Set Utilization

---

### Q1. What is utilization in floorplanning and how is it defined? (NVIDIA, Intel)

*Asked at: NVIDIA, Intel, Qualcomm*

Utilization is the ratio of total cell area (standard cells + macros) to the total core area, expressed as a percentage:  $\text{Utilization} = (\text{Total Cell Area} / \text{Core Area}) * 100\%$ . It represents how densely the chip core is populated with logic. At 100% utilization, no space remains for routing, clock trees, or optimization buffers — making the design unimplementable. Practical utilization leaves adequate white space. Setting utilization is a fundamental floorplanning decision that directly determines core area size and has cascading effects on routing, timing, and power.

---

### Q2. What happens if utilization is set too high or too low? (STMicroelectronics, AMD)

*Asked at: STMicroelectronics, AMD, Cadence*

Too high utilization (e.g., >85%) causes: routing congestion (insufficient routing tracks), inability to insert optimization buffers, difficulty meeting timing (no room for clock tree buffers), routing DRC violations, and potentially un-routable designs. Too low utilization (e.g., <50%) wastes silicon area, increases die cost, lengthens wires (increasing delay and power), and reduces packaging density. Optimal utilization balances area efficiency against implementation flexibility. Teams iterate on utilization during floorplanning by analyzing congestion maps and timing results, adjusting until both area and timing targets are achievable.

---

### Q3. How does process node affect utilization targets? (Google, TSMC)

*Asked at: Google, TSMC, Samsung*

Advanced process nodes (7nm, 5nm, 3nm) have tighter design rules, more complex double/multiple patterning requirements, and reduced routing pitch, making routing more difficult per unit area. These factors necessitate lower target utilization (60-70%) compared to older nodes (180nm, 65nm) where 80-85% was achievable. Advanced nodes also have larger standard cell heights relative to routing pitch, leaving fewer routing tracks per cell row. Additionally, FinFET and gate-all-around transistor rules create more complex keep-out zones, further reducing effective utilization capacity compared to planar CMOS processes.

---

### Q4. What is effective utilization vs. placement utilization? (Broadcom, Apple)

*Asked at: Broadcom, Apple, Synopsys*

Placement utilization counts only the area occupied by cells relative to core area. Effective utilization accounts for all area consumers including: macro keep-out halos, power stripes (metal consumed by power grid), routing blockages, voltage domain boundaries, and physical-only cells (tap cells, filler cells, end caps). Effective utilization is always higher than cell-only utilization. A design targeting 75% placement utilization may have 85-90% effective utilization when all physical-only structures are counted. EDA tools report both metrics; effective utilization is the more accurate predictor of routing congestion.

---

---

## 5. Aspect Ratio (AR)

---

### **Q1. What is aspect ratio in floorplanning and what values are commonly used? (Intel, STMicroelectronics)**

*Asked at: Intel, STMicroelectronics, Cadence*

Aspect ratio (AR) is defined as the ratio of floorplan height to floorplan width ( $AR = H/W$ ). A value of 1.0 produces a square floorplan. Values between 0.5 and 2.0 are common in practice. AR is chosen during floorplanning based on: IO pad distribution requirements, macro shapes and natural arrangement, package constraints, and routing balance. A square die ( $AR=1$ ) is preferred when possible for balanced routing and manufacturing yield. Extreme AR values (e.g., 4:1 or 1:4) create very elongated dies with unbalanced routing resources and longer critical paths.

---

### **Q2. How does aspect ratio affect clock tree synthesis and timing? (NVIDIA, Qualcomm)**

*Asked at: NVIDIA, Qualcomm, AMD*

Aspect ratio directly impacts clock tree wire length and achievable skew. A square die allows a symmetric H-tree clock topology with balanced path lengths. An elongated die (high or low AR) forces the clock tree to drive sinks across a long dimension, increasing clock insertion delay and making skew balancing more difficult. Long clock wires increase capacitance, power consumption, and susceptibility to on-chip variation (OCV), which degrades clock uncertainty. Timing-critical designs prefer near-square AR to minimize clock distribution challenges and reduce the number of clock tree buffer levels needed for skew control.

---

### **Q3. What is the relationship between AR and macro placement? (Google, Apple)**

*Asked at: Google, Apple, Broadcom*

AR strongly influences macro placement strategy. Large memory arrays (SRAMs) are often rectangular with 2:1 or 4:1 internal AR. Aligning the die AR to accommodate dominant macro shapes reduces wasted space and routing complexity. If a design has multiple tall SRAMs, a taller die ( $AR > 1$ ) may allow more efficient macro arrangement. Conversely, wide datapaths favor wider dies. The natural AR of the dominant macros often guides the overall die AR selection during initial floorplanning exploration. Poor AR-to-macro-shape matching forces macros into awkward orientations, wasting area and creating routing congestion.

---

### **Q4. How is AR adjusted during floorplan iterations and what drives changes? (AMD, Samsung)**

*Asked at: AMD, Samsung, MediaTek*

AR is adjusted during floorplan iterations when: congestion analysis reveals routing imbalance (more horizontal than vertical congestion suggests the die is too wide), timing analysis shows long clock paths across one dimension, macro placement results in large wasted areas (suggesting a different AR better fits the macro arrangement), or package constraints change. Typically 3-5 AR values are explored in early floorplan iterations using coarse placement and global routing analysis to evaluate congestion and timing. The AR that gives the best balance of congestion, timing, and area efficiency is selected for detailed floorplanning.

---

## 6. Create Core Boundary

---

### Q1. What is the core boundary in floorplanning and how is it defined? (Intel, Cadence)

*Asked at: Intel, Cadence, Synopsys*

The core boundary (also called the core ring or placement boundary) is the rectangular region inside the die where all standard cells, macros, and routing are placed. It is defined by: core offset from die edges (space reserved for IO ring, power rings, and corner structures), core width and height derived from core area and aspect ratio, and alignment to placement site grid (core dimensions must be integer multiples of the site width and row height). The core boundary is the first physical constraint established in floorplanning and all subsequent steps operate within it.

---

### Q2. What is the core-to-IO spacing and what determines it? (NVIDIA, STMicroelectronics)

*Asked at: NVIDIA, STMicroelectronics, TSMC*

Core-to-IO spacing is the distance between the core boundary edge and the IO cell placement region. It accommodates: power ring routing (VDD/VSS rings around the core), signal routing from IO pads to core logic, ESD clamp cells, level shifters, and bondwire or bump landing areas. Typical spacing is 50-200 micrometers depending on process node and power requirements. Advanced nodes with many power domains may require wider IO-to-core spacing for multiple power rings. Flip-chip designs have different requirements since bumps are distributed across the die surface rather than at the periphery.

---

### Q3. How does the core boundary affect placement row creation? (AMD, Google)

*Asked at: AMD, Google, Broadcom*

The core boundary defines the extent of placement rows. Placement rows are horizontal bands (each row height equals the standard cell height) that fill the entire core area from bottom to top. The core boundary's width must be an integer multiple of the site width (typically 0.2-0.4 micrometers) so rows align perfectly with the boundary. The core boundary's height must be an integer multiple of the row height. Any mismatch creates partial rows that cannot accommodate standard cells. EDA tools automatically create rows within the core boundary during floorplan creation, and macros placed inside the core remove rows from their footprint area.

---

### Q4. What is the difference between core area, core boundary, and placement area? (Apple, Samsung)

*Asked at: Apple, Samsung, Marvell*

Core area is the total rectangular area enclosed by the core boundary, measured as width x height. Core boundary is the geometric perimeter (rectangle) defining the core edge — a physical constraint. Placement area is the effective area within the core where standard cells can actually be placed, excluding: macro footprints, placement blockages, IO buffer placement regions, voltage domain boundary margins, and keep-out zones. Placement area is always less than core area. The ratio of actual cell area to placement area gives the effective placement density, which must remain within routing-feasible limits (typically below 80-85%).

---

## 7. Place IO Pads

---

### Q1. What are IO pads and what is the strategy for placing them in floorplanning? (Intel, Qualcomm)

*Asked at: Intel, Qualcomm, STMicroelectronics*

IO pads are interface cells at the die periphery providing electrical connection between internal logic and the package (wire bonds or bumps). IO pad placement strategy considers: functional grouping (related signals placed together to minimize internal routing), timing requirements (high-speed interfaces like DDR, PCIe placed close to their associated controllers), power distribution (power/ground pads distributed uniformly for IR drop management), signal integrity (differential pairs placed adjacently with proper spacing), and package pin assignment (IO pad locations must match package ball/pin layout). Poor IO placement forces long internal routing, degrading timing and congestion.

---

### Q2. What is the difference between wire-bond and flip-chip IO pad placement? (NVIDIA, AMD)

*Asked at: NVIDIA, AMD, Broadcom*

Wire-bond IO pads are placed only at the die periphery (pad ring), with signal connections made via bond wires from pad to package lead frame. This limits IO count to peripheral perimeter. Flip-chip IO uses C4 (Controlled Collapse Chip Connection) bumps distributed across the entire die surface, dramatically increasing IO count and reducing inductance. Flip-chip pad placement is more flexible (bumps anywhere on die) but requires careful bump assignment to RDL (redistribution layer) routing. Flip-chip also enables better power distribution (power bumps throughout die) and higher bandwidth interfaces than wire-bond designs.

---

### Q3. How do IO pad locations affect internal signal routing and congestion? (Google, Apple)

*Asked at: Google, Apple, Samsung*

IO pad locations directly determine where signals enter and exit the core, creating forced routing paths. If an IO pad for a high-speed bus is placed on the opposite side of the die from its associated controller block, all bus signals must be routed across the entire die, consuming routing tracks and potentially creating congestion corridors. Proper IO placement minimizes the distance between IO pads and their logically connected core blocks. Analyzing flyline connectivity (straight-line connections between IO pads and core logic) during IO placement helps identify poor assignments before detailed routing begins.

---

### Q4. What are pad-limited vs. core-limited designs and how does IO placement differ? (Broadcom, MediaTek)

*Asked at: Broadcom, MediaTek, Marvell*

In pad-limited designs, the die size is determined by the IO pad count and pitch — the perimeter needed to accommodate all pads is larger than what the core logic area requires. IO placement must fit all pads in the available perimeter, often requiring double-row pad staggering or flip-chip migration. In core-limited designs, logic area determines die size, and

there is ample perimeter space for IO pads with flexibility in their placement. Pad-limited designs are common in high-IO-count chips (microcontrollers, CPUs), while core-limited designs are typical for complex SoCs or memory-dominated chips with relatively few IOs.

---

## 8. Place Macros

---

### Q1. What are the key principles of macro placement in floorplanning? (NVIDIA, Intel)

*Asked at: NVIDIA, Intel, Qualcomm*

Key macro placement principles: (1) Abutment to core boundary walls — place macros along core edges to minimize wasted space and simplify routing channel creation. (2) Connectivity-driven placement — macros should be close to their primary interface logic. (3) Avoid macro islands — don't isolate macros in the middle of the core, which creates routing bottlenecks. (4) Consider macro orientation — flip/rotate to align power pins with power stripes. (5) Leave adequate routing channels between macros. (6) Respect timing requirements — SRAM macros used in critical paths should be placed near timing-sensitive logic. (7) Balance placement — avoid all macros on one side.

---

### Q2. Why are macros placed against core boundaries rather than in the center? (STMicroelectronics, AMD)

*Asked at: STMicroelectronics, AMD, Cadence*

Placing macros against core boundaries (walls) is preferred because: it maximizes contiguous standard cell placement area in the center, creates natural routing channels between macros and core edges that are accessible from all directions, avoids macros blocking routing paths between different core regions, enables power stripe routing along the routing channels parallel to macro edges, and prevents the macro from splitting the standard cell area into disconnected islands that are harder to route. Center-placed macros fragment the core into separate regions with limited connectivity, dramatically increasing congestion and routing complexity.

---

### Q3. What is macro snapping and why is it important? (Google, Apple)

*Asked at: Google, Apple, Broadcom*

Macro snapping aligns macro placement to specific grid points — typically the power stripe pitch and placement row boundaries. Macros must be aligned so that their power/ground pins align with the power distribution grid (power stripes), ensuring low-resistance power connections. Mis-aligned macros require jogs in power stripes that reduce current carrying capacity and create IR drop hotspots. Macros must also align to row boundaries so standard cells can abut them without leaving unusable fractional rows. Modern EDA tools enforce snapping through macro placement constraints, and violations result in power DRC errors during signoff.

---

### Q4. How does the number of macros affect floorplanning complexity and time? (AMD, Samsung)

*Asked at: AMD, Samsung, TSMC*

Floorplanning complexity scales super-linearly with macro count. With  $N$  macros, there are  $N!$  possible arrangements (ignoring orientation), making exhaustive search impossible for large  $N$ . Designs with 10-20 macros can be manually floorplanned with good results. 50-100 macros require automated macro placement algorithms (simulated annealing, force-directed, or ML-based approaches like Google's chip floorplanning using reinforcement learning). 200+ macros require hierarchical approaches. Each additional macro adds potential for creating routing blockages, connectivity conflicts, and power grid complexity, exponentially increasing the time needed to achieve a legal, routable floorplan.

---

## 9. Macro Clustering

---

### Q1. What is macro clustering and when is it applied in floorplanning? (Intel, STMicroelectronics)

*Asked at: Intel, STMicroelectronics, Cadence*

Macro clustering groups multiple related macros together based on their functional relationship and connectivity. Highly connected macros (e.g., a processor core with its L1 cache SRAMs, or a DMA controller with its buffer memories) are clustered into a region, keeping their interconnections short and reducing routing congestion. Clustering is applied during initial macro placement planning before detailed macro positioning. It simplifies the floorplanning problem by treating clusters as units, and ensures that macros sharing many nets are physically co-located, improving timing and reducing the wire area consumed by inter-macro routing.

---

### Q2. What algorithms are used for macro clustering? (NVIDIA, Qualcomm)

*Asked at: NVIDIA, Qualcomm, Synopsys*

Common macro clustering algorithms include: connectivity-based clustering (graph partitioning using net weights proportional to bus width or timing criticality — macros with more connections are merged), hierarchical clustering (bottom-up merging of nearest neighbors by connectivity strength), and k-means or spectral clustering on the netlist graph. Simulated annealing and genetic algorithms explore cluster-to-region assignments. Modern tools use machine learning approaches where floorplan quality metrics (wirelength, timing, congestion) train models that predict optimal cluster groupings. Cadence Innovus and Synopsys Fusion Compiler include automated macro clustering as part of their floorplanning engines.

---

### Q3. How does macro clustering reduce routing congestion? (Google, AMD)

*Asked at: Google, AMD, Broadcom*

Macro clustering reduces congestion by keeping high-bandwidth connections between related macros extremely short, minimizing the number of long cross-chip nets that consume routing tracks over wide areas. A clustered memory subsystem (processor + cache SRAMs) co-located in one corner means data bus connections are short and contained within that corner's routing area, rather than spanning the die. This frees routing resources in other regions for signal routes. Clustering also enables better power domain assignment (related macros often share power domains), reducing level shifter count and simplifying UPF

implementation.

---

#### **Q4. What are the risks of over-clustering macros in a floorplan? (Apple, Samsung)**

*Asked at: Apple, Samsung, MediaTek*

Over-clustering creates large macro-dense regions that become routing bottlenecks and reduce placement flexibility. If too many macros are forced into a small area, there may be insufficient space for standard cells that must be nearby, forcing them to be placed far away and lengthening critical paths. Large clusters of macros can also block power stripe routing, creating IR drop islands with inadequate power supply. Additionally, overly tight clustering leaves no room for optimization buffers between macros, making timing closure difficult. The optimal clustering balances connectivity benefits against the need for adequate routing channels and standard cell placement space around each macro group.

---

## **10. Flyline Analysis**

---

#### **Q1. What are flylines in floorplanning and what do they indicate? (NVIDIA, Intel)**

*Asked at: NVIDIA, Intel, Qualcomm*

Flylines (also called ratsnest lines or logical connections) are straight lines drawn between connected cells/macros showing logical net connections before actual routing. They represent the shortest possible wire path between connected endpoints. Flyline analysis visualizes: total wire demand across the chip, crossing patterns that indicate routing congestion, long-distance connections that will be timing-critical, high-fanout nets (many lines emanating from one point), and inter-block connectivity density. Dense, crossing flyline regions predict routing congestion hotspots. A good floorplan has mostly short, non-crossing flylines indicating that connected logic is physically co-located.

---

#### **Q2. How is flyline analysis used to improve macro placement? (STMicroelectronics, AMD)**

*Asked at: STMicroelectronics, AMD, Cadence*

After initial macro placement, flyline analysis reveals suboptimal arrangements: if flylines between two macros all cross a third macro's boundary, those macros need to be repositioned to eliminate the crossing. Macros with dense flylines between them should be moved closer together. Flylines crossing the entire die suggest macros should be swapped or mirrored. Engineers iteratively reposition macros, re-analyze flylines, and refine placement until flylines are mostly short and non-crossing. Minimizing total flyline length (equivalent to minimizing HPWL) is the primary metric for evaluating macro placement quality during floorplanning.

---

#### **Q3. What is the relationship between flyline length and actual routed wirelength? (Google, Broadcom)**

*Asked at: Google, Broadcom, Apple*

Flyline length (Half-Perimeter Wirelength, HPWL) provides a lower bound estimate of actual routed wirelength. The actual routed wire is always longer than the flyline because: routes

must follow Manhattan geometry (only horizontal and vertical segments), routing must avoid macro obstacles and blockages causing detours, congestion forces routes through alternative paths, and via connections between metal layers add vertical wire length. Typically, actual routed wirelength is 1.1-1.3x the total flyline length for well-floorplanned designs. Congested designs with many detours can have actual wirelength 1.5-2x or more the flyline estimate.

---

#### **Q4. What tools and commands are used to visualize and analyze flylines? (Synopsys, Cadence)**

*Asked at: Synopsys, Cadence, Mentor*

In Synopsys ICC2/Fusion Compiler, flylines are visualized using the GUI's net flyline display mode or commands like `report_net` with physical information. Cadence Innovus displays flylines interactively and provides commands like `reportCongestion` and `analyzeFloorplan` for quantitative analysis. Engineers typically: display all flylines in the GUI to visually inspect crossing patterns, filter flylines by timing criticality or bus width to focus on important connections, use congestion heat maps alongside flylines to correlate predicted congestion with flyline density, and run global routing estimation (`reportGlobalRouteData`) to get quantitative congestion predictions based on flyline analysis.

---

## **11. Macro Optimization**

---

#### **Q1. What optimizations are performed on macro placement after initial floorplanning? (NVIDIA, Intel)**

*Asked at: NVIDIA, Intel, Qualcomm*

Post-initial macro placement optimizations include: (1) Macro orientation flipping/rotation to align power pins with power stripes and reduce routing detours. (2) Incremental repositioning to reduce flyline crossings identified in flyline analysis. (3) Timing-driven macro repositioning moving timing-critical macros closer to their interface logic. (4) Congestion-driven spreading to open routing channels in congested regions. (5) Adjusting macro-to-macro spacing to ensure adequate routing channels. (6) Evaluating alternate macro arrangements using global routing estimation to predict congestion. (7) Checking and fixing power grid connectivity after position changes.

---

#### **Q2. What is automated macro placement and how does it work? (Google, AMD)**

*Asked at: Google, AMD, Broadcom*

Automated macro placement uses optimization algorithms to find good macro positions without manual trial-and-error. Simulated annealing is the classical approach: starting from an initial (possibly random) placement, random moves (translate, rotate, swap) are accepted based on a cost function (HPWL + congestion estimate) and temperature parameter that decreases over time. Modern approaches include force-directed methods (macros repel each other, are attracted to connected cells) and reinforcement learning (Google's work trained RL agents that outperformed human engineers on macro placement). Automated tools typically produce solutions in minutes that would take human engineers days to achieve manually.

---

### Q3. How does macro orientation affect routing and power distribution? (STMicroelectronics, Apple)

*Asked at: STMicroelectronics, Apple, Samsung*

Macro orientation (0, 90, 180, 270 degrees, and flipped variants — 8 possible orientations) determines where macro pins are located on its boundary. Choosing orientation aligns: power/ground pins (VDDCE, VSSE) with horizontal/vertical power stripes for minimum resistance connections, data/address ports toward the side facing their connected controller logic (minimizing routing detours), and the macro body flush with the core boundary or adjacent macros for clean abutment. Wrong orientation forces power connections through long jog routes with high resistance causing IR drop, and signal pins facing away from their logical connections creating long detour routes and timing violations.

---

### Q4. What is the role of timing analysis in macro optimization? (Qualcomm, TSMC)

*Asked at: Qualcomm, TSMC, Cadence*

Timing analysis drives macro optimization by identifying which macro-to-macro or macro-to-logic paths are timing-critical. If STA reveals a critical path passing through a specific SRAM macro and its associated datapath logic, those elements must be placed closer together to reduce wire delay. Macros on the critical path receive higher placement priority and may override other considerations (power or congestion) to minimize their interface wire length. Incremental timing analysis during macro optimization — re-running STA after each macro repositioning — provides feedback on whether changes improve the critical path. Tools like Tempus or PrimeTime can be invoked inline during macro optimization.

---

## 12. Channel Spacing

---

### Q1. What are routing channels in floorplanning and how is their spacing determined? (Intel, STMicroelectronics)

*Asked at: Intel, STMicroelectronics, Synopsys*

Routing channels are gaps between macros or between macros and core boundaries where signal and power routes pass. Channel spacing is the minimum gap required to accommodate all expected routes through that channel. Spacing is determined by: number of signals expected to route through the channel (based on connectivity analysis), available metal layers in that region, routing track pitch (metal spacing rules), presence of power stripes (which consume routing tracks), and via landing requirements. Minimum channel width = (number of nets / available tracks per layer) \* metal pitch. Inadequate channel spacing creates routing congestion that may block routing completion.

---

### Q2. What happens if channels between macros are too narrow? (NVIDIA, Qualcomm)

*Asked at: NVIDIA, Qualcomm, AMD*

Narrow channels between macros force all inter-area routing through a restricted bottleneck, creating severe routing congestion. Consequences include: routing overflow in the channel (more routing demand than available tracks), rerouting of nets through longer detour paths

increasing wirelength and delay, potential routing DRC violations (shorts, minimum spacing violations from forced dense routing), timing degradation from longer routes on critical paths, and in worst cases complete routing failure (open circuits). The affected channels appear as bright red hotspots in congestion heat maps. Resolution requires widening channels by moving macros apart, adding routing bypass layers, or reducing connectivity through the channel.

---

### **Q3. How do power stripes affect available channel routing capacity? (Google, Apple)**

*Asked at: Google, Apple, Broadcom*

Power stripes (wide metal connections for VDD/VSS distribution) run through routing channels and consume metal tracks that would otherwise be available for signal routing. A channel with 20 available metal tracks per layer may have 4-6 tracks occupied by power stripes on each metal layer, reducing available signal routing capacity by 20-30% or more. In multi-voltage designs with multiple power domains, multiple power stripe sets (one per domain) can consume a large fraction of channel routing capacity. Floorplanning must account for power stripe area when estimating channel routing capacity, and channels must be widened proportionally to compensate for tracks consumed by power distribution.

---

### **Q4. What is the minimum recommended channel width between two large SRAMs? (AMD, Samsung)**

*Asked at: AMD, Samsung, MediaTek*

Minimum channel width between large SRAMs depends on: address/data bus width (a 64-bit data bus requires routing 64 signal tracks plus control signals), metal pitch at the process node (7nm may have 28nm track pitch on lower metals), number of metal layers available for routing in the channel, power stripe width and pitch, and timing requirements (wider channels with shorter jogs may be needed for timing). As a rough rule, channels should accommodate at least 2-3x the primary bus width in routing tracks, plus power stripes. For a 64-bit bus at 7nm with 28nm pitch, this suggests minimum channel widths of 5-10 micrometers excluding power stripes.

---

## **13. Halo Creation**

---

### **Q1. What is a macro halo and why is it created around macros? (Intel, NVIDIA)**

*Asked at: Intel, NVIDIA, Qualcomm*

A macro halo (also called a keep-out zone or exclusion zone) is a rectangular boundary surrounding a macro within which standard cells cannot be placed. Halos are created to: ensure routing access to macro pins without congestion from abutting standard cells, provide space for macro decoupling capacitors, accommodate antenna diodes at macro input pins, prevent latch-up between macro and standard cell transistors (substrate isolation), and allow routing channels for signals entering/exiting macro ports. Typical halo sizes are 1-10 micrometers depending on process node. Halos are defined in the DEF or through EDA tool constraints.

---

## Q2. What is the difference between a hard halo and a soft halo? (STMicroelectronics, AMD)

*Asked at: STMicroelectronics, AMD, Cadence*

A hard halo strictly prevents any standard cell placement within the halo boundary — it is an absolute keep-out zone enforced by the placer. The placement engine will not place cells in this region under any circumstances. A soft halo is a preference rather than a hard constraint — the placer tries to avoid placing cells in the soft halo area but may violate it if necessary to achieve timing or legality requirements. Hard halos are used when the exclusion is mandated by electrical requirements (latch-up prevention, antenna rules). Soft halos are used for routing convenience where minor violations can be tolerated.

---

## Q3. How does halo size affect floorplan area and routability? (Google, Apple)

*Asked at: Google, Apple, Broadcom*

Larger halos increase the effective area consumed by each macro (macro area + halo area on all sides), reducing available standard cell placement area and potentially forcing utilization adjustment. For a macro with 5-micrometer halos on all sides, the effective area increase can be significant for small macros. However, larger halos provide better routing access, easier antenna fix insertion, and more reliable latch-up prevention. Too-small halos may cause DRC violations near macro pins or antenna violations on macro input nets. Halo size is a tradeoff between area efficiency and routing/reliability margin, typically set conservatively for first-pass floorplanning.

---

## Q4. What is the relationship between halos and antenna violations? (Qualcomm, TSMC)

*Asked at: Qualcomm, TSMC, Samsung*

Antenna violations occur when long metal routes accumulate charge during plasma etching, potentially damaging gate oxide of connected transistors. Macro input pins are particularly susceptible since they connect to gate oxides of internal transistors. Halos around macros provide space to insert antenna diodes directly on the routing approaching macro input pins. Without adequate halo space, antenna diodes cannot be placed close to the macro pins, requiring longer detour routes for antenna fix which may themselves create antenna violations. Proper halo sizing ensures antenna diodes can be placed within a few micrometers of each macro input pin, providing effective protection with minimal routing overhead.

---

# 14. Routing Channel Creation

---

## Q1. What is a routing channel and how is it created during floorplanning? (Intel, STMicroelectronics)

*Asked at: Intel, STMicroelectronics, Cadence*

A routing channel is a dedicated corridor of space (free of macros and standard cell placement) reserved for signal and power routing between different regions of the chip. Routing channels are created during floorplanning by deliberately spacing macros apart or away from core boundaries to leave gaps. The channel width is sized based on expected routing demand (number of nets crossing the channel multiplied by routing track pitch).

Channels are created both horizontally and vertically to ensure routing connectivity in both Manhattan directions. Power stripes run along routing channels, so their width must accommodate both power and signal routing needs simultaneously.

---

### **Q2. How do you estimate the required width of a routing channel? (NVIDIA, Qualcomm)**

*Asked at: NVIDIA, Qualcomm, AMD*

Required channel width estimation: (1) Count the number of nets expected to cross the channel (from connectivity analysis or flyline density). (2) Divide by the number of available routing metal layers in that region. (3) Multiply by the metal routing pitch for each layer. (4) Add space for power stripes (width x number of stripes crossing the channel). (5) Add margin (20-30%) for routing flexibility and detours. Example: 100 nets crossing, 4 signal layers, 0.032um pitch =  $100/4 * 0.032\text{um} = 0.8\text{um}$  per layer total. With 5um power stripes and 30% margin, total channel width approximately 5-10um minimum.

---

### **Q3. What is the difference between global routing channels and local routing channels? (Google, AMD)**

*Asked at: Google, AMD, Broadcom*

Global routing channels are major corridors spanning large portions of the chip, enabling connectivity between major functional blocks (processor to memory, IO to core logic). They must accommodate hundreds to thousands of nets and wide power buses. They are planned during top-level floorplanning and determine the overall chip routing topology. Local routing channels are smaller gaps between individual macros within a functional block, handling connectivity within that block. Local channels accommodate tens to hundreds of nets. Global channels use upper metal layers (lower resistance, wider pitch), while local channels primarily use lower metal layers for fine-pitch signal routing.

---

### **Q4. What happens if routing channels are not properly created during floorplanning? (Apple, Samsung)**

*Asked at: Apple, Samsung, TSMC*

Insufficient routing channels cause cascading failures: the router cannot complete all connections (open circuit violations), nets are rerouted through extremely long detours increasing wirelength 2-5x (dramatically hurting timing and power), congestion in narrow channels causes DRC violations (shorts between parallel routes), power supply routing is compromised reducing current delivery and increasing IR drop, and the overall routing runtime increases dramatically as the tool struggles to find feasible routes. In severe cases, the design is simply unroutable and requires complete re-floorplanning. This is why routing channel creation is a critical floorplanning step — routing problems are very expensive to fix late in the design cycle.

---

## **15. Placement Blockages**

---

### **Q1. What are placement blockages and what are the different types? (Intel, NVIDIA)**

*Asked at: Intel, NVIDIA, Qualcomm*

Placement blockages prevent standard cells from being placed in specified regions. Types include: (1) Hard blockage — no cells allowed, ever, in this region. (2) Soft blockage — cells avoided here but placer may use space if necessary for legality. (3) Partial blockage — cells allowed but at reduced density (e.g., maximum 50% utilization in this region). (4) Buffer blockage — only buffer/inverter cells blocked (other logic allowed). (5) Cell type blockage — specific cell types excluded (e.g., no flip-flops near analog blocks). Blockages are typically rectangular regions defined in the DEF file or through EDA tool constraint commands.

---

## **Q2. Why are placement blockages created near analog macros and PLLs? (STMicroelectronics, AMD)**

*Asked at: STMicroelectronics, AMD, Google*

Analog blocks (PLLs, ADCs, DACs, bandgap references) are highly sensitive to substrate noise, power supply noise, and magnetic coupling. Standard cells switching nearby inject noise through the substrate and power rails that can degrade analog performance (increased jitter in PLLs, reduced SNR in ADCs). Placement blockages around analog macros enforce a noise isolation zone where digital switching is prohibited. The blockage distance is determined by the analog block's noise sensitivity specification and the substrate coupling characteristics of the process. Additionally, blockages prevent routing congestion directly adjacent to sensitive analog pins where controlled impedance routing is needed.

---

## **Q3. What is a routing blockage and how does it differ from a placement blockage? (Apple, Broadcom)**

*Asked at: Apple, Broadcom, Cadence*

A placement blockage restricts where standard cells can be physically placed in the design, operating at the cell/row level. A routing blockage restricts where metal wires can be routed, operating at the metal layer level. Routing blockages can be layer-specific (e.g., no routing on Metal 1-3 over an analog block) while placement blockages prevent cell placement regardless of metal layers. Both types are used together: a placement blockage creates cell-free space near a macro, while layer-specific routing blockages on upper layers prevent long horizontal bus routes from passing over noise-sensitive analog regions. They complement each other for complete physical isolation.

---

## **Q4. How do placement blockages interact with spare cell insertion? (Qualcomm, Samsung)**

*Asked at: Qualcomm, Samsung, Synopsys*

Placement blockages directly reduce available space for spare cells since spare cells are standard cells and cannot be placed in hard or soft blockage regions. This can create situations where ECO coverage (the ability to implement metal-only ECOs using spare cells) is poor in regions with extensive placement blockages. Design teams must carefully balance blockage extent against ECO coverage requirements. Solutions include: using narrower blockages only where strictly necessary, placing ECO-critical spare cells before creating blockages, using targeted spare cell clusters at blockage boundaries where they can service both the blockage-protected region and surrounding standard cell areas through short metal connections.

---

## 16. Check Congestion

---

### Q1. How is congestion checked during floorplanning before actual placement? (NVIDIA, Intel)

*Asked at: NVIDIA, Intel, Qualcomm*

Floorplan congestion is estimated using global routing analysis without actual cell placement. The tool divides the core into routing tiles (GRC bins, typically 10-50 micrometers per bin), estimates routing demand in each bin based on flyline density and macro pin locations, compares demand against routing supply (available tracks per bin), and reports overflow (demand > supply). Tools like Innovus (reportCongestion) or Fusion Compiler (route\_global) run a fast global routing step to generate congestion heat maps. Red/orange tiles indicate problematic regions requiring floorplan modification before proceeding to placement. This early check prevents wasted effort on a fundamentally unroutable floorplan.

---

### Q2. What is routing overflow and how is it quantified? (STMicroelectronics, AMD)

*Asked at: STMicroelectronics, AMD, Cadence*

Routing overflow is the excess routing demand beyond available routing supply in a specific routing tile. It is quantified as:  $\text{Overflow} = \max(0, \text{Demand} - \text{Supply})$ . Total overflow = sum of overflow across all tiles. Overflow is also expressed as a percentage:  $(\text{total overflow nets} / \text{total routed nets}) * 100\%$ . A floorplan is considered congestion-clean when total overflow is 0% or very close to 0% (some tools accept up to 0.1-0.5% for complex designs). Overflow greater than 1-2% indicates the design may not route completely. The GRC (Global Route Congestion) map visually shows overflow using color coding: green = no overflow, yellow = marginal, red = significant overflow.

---

### Q3. What are the most common causes of congestion hotspots in floorplanning? (Google, Apple)

*Asked at: Google, Apple, Broadcom*

Common congestion hotspot causes: (1) Macro placement creating bottlenecks — two macros too close, forcing all routes through a narrow gap. (2) High-connectivity regions — blocks with many inter-connections creating local routing demand exceeding supply. (3) Power stripe density too high — power stripes consuming excessive routing tracks in signal routing layers. (4) IO placement mismatches — IO pads far from their logic forcing long bus routes across congested regions. (5) Insufficient channel width — routing channels undersized for the demand. (6) Missing routing layers — blockages preventing use of critical routing layers in congested areas. (7) High utilization pockets where cell density is locally too high.

---

### Q4. What changes are made to the floorplan to fix congestion identified during floorplan checks? (AMD, Samsung)

*Asked at: AMD, Samsung, TSMC*

Congestion fixes at the floorplan stage include: (1) Move macros to widen channels in congested regions. (2) Reduce utilization in congested areas by enlarging the core or redistributing cells. (3) Add routing bypass paths (dedicated metal layers freed from power stripes in congested regions). (4) Reposition IO pads to reduce long cross-chip signal routes.

(5) Split large macros into smaller ones if possible (for memory compiler macros). (6) Add placement blockages to prevent cell density buildup in already-congested areas. (7) Rotate/flip macros to relocate high-pin-density port faces toward available routing space. (8) Adjust power stripe pitch to free more signal routing tracks in overflow regions.

---

## 17. Check Macro Overlaps

---

### Q1. What are macro overlaps and why must they be resolved before proceeding? (Intel, NVIDIA)

*Asked at: Intel, NVIDIA, STMicroelectronics*

Macro overlaps occur when two or more macros are placed such that their physical boundaries intersect, occupying the same silicon area. Since each macro is a hard block occupying real silicon, overlaps are physically impossible to implement and represent a fundamental floorplan error. They must be resolved because: overlapping macros cannot both be manufactured, the placement tool cannot legally place standard cells in overlapping regions, routing cannot be performed in the overlap area, and power grid planning fails for overlapping regions. Macro overlaps are a hard stop — no subsequent design steps (placement, routing) can proceed with macro overlaps present.

---

### Q2. What causes macro overlaps and how are they detected? (Qualcomm, AMD)

*Asked at: Qualcomm, AMD, Cadence*

Macro overlaps are caused by: manual placement errors (wrong coordinates entered), automated placement algorithms not converging to a legal solution, macros being resized (larger version chosen after initial placement), floorplan modifications (core boundary shrunk, compressing macros), copy-paste errors in DEF files, and importing macros from different technology nodes without updating footprints. Detection methods: EDA tools run legality checks (check\_legality in Innovus, check\_placement in ICC2) reporting overlap pairs with overlap area and coordinates. Visual inspection in the GUI shows overlapping macros highlighted in error colors. LVS-style physical checks also flag overlapping instances.

---

### Q3. How are macro overlaps resolved — what is the resolution strategy? (Google, Apple)

*Asked at: Google, Apple, Broadcom*

Macro overlap resolution strategies: (1) Manual repositioning — move one or both overlapping macros to non-overlapping positions, prioritizing less connected macros for movement. (2) Automated legalization — use the floorplan tool's macro legalization feature that resolves overlaps with minimum displacement. (3) Core boundary expansion — if the core is too small to fit all macros without overlap, increase core size (adjust utilization or die dimensions). (4) Macro elimination — use a different memory architecture with fewer/smaller macros if possible. (5) Hierarchical partitioning — move some macros into a separate voltage area or hierarchical block, reducing macro density in the parent level.

---

### Q4. What DRC checks verify macro placement legality beyond overlap checking? (Samsung, TSMC)

*Asked at: Samsung, TSMC, Synopsys*

Beyond overlap checking, macro placement DRC checks include: (1) Macro-to-macro spacing (some macros require minimum spacing beyond zero for well isolation or routing access). (2) Macro alignment to placement grid (macros must sit on legal x/y coordinates aligned to row and site grid). (3) Macro-to-boundary spacing (minimum distance from macro edge to core boundary for routing). (4) Power pin connectivity (macro VDD/VSS pins must align with power stripes within tolerance). (5) Orientation legality (some macros have restricted valid orientations). (6) Halo overlap check (halos of different macros should not overlap, or only soft halos should overlap). (7) Voltage domain check (macros must be inside their correct voltage region).

---

## 18. Floorplan Verification

---

### Q1. What does floorplan verification check and what are the signoff criteria? (Intel, NVIDIA)

*Asked at: Intel, NVIDIA, Qualcomm*

Floorplan verification comprehensively checks: legality (no overlaps, all cells on grid), congestion (global routing overflow within budget), timing (pre-placement timing estimation meets target with adequate margin), power integrity (IR drop estimation acceptable, power grid coverage sufficient), DRC (all physical rules satisfied — spacing, width, enclosure for power rings/stripes), connectivity (all macro pins accessible for routing), utilization (target met without exceeding maximum), IO placement correctness, voltage domain integrity (all cells in correct power domains), and floorplan vs. netlist consistency. Signoff criteria require all checks passing or documented waivers for acceptable deviations.

---

### Q2. What is IR drop analysis at the floorplan stage and why is it important? (STMicroelectronics, AMD)

*Asked at: STMicroelectronics, AMD, Broadcom*

Floorplan-stage IR drop analysis estimates voltage drop across the power distribution network before detailed routing. It uses the power grid topology (ring and stripe geometry, metal layer and width), estimated current demand from power analysis (activity-based or average current estimation from synthesis), and metal resistance models to calculate voltage drop at each grid node. If IR drop exceeds specification (typically less than 5-10% of VDD at any node), the power grid must be strengthened by adding more stripes, widening existing stripes, or adding more power pads. Early IR drop analysis prevents discovering power problems after routing is complete, which is extremely costly to fix.

---

### Q3. What timing checks are included in floorplan verification? (Google, Apple)

*Asked at: Google, Apple, Samsung*

Floorplan timing verification includes: estimated timing using wireload models or physical estimation (before actual placement), critical path identification to ensure timing-critical paths have co-located logic in the floorplan, macro timing interface checks (SRAM access time compatibility with clock period), IO timing estimation (input/output delay budget feasibility), clock distribution feasibility (can a clock tree be synthesized to all sinks within skew budget given floorplan dimensions), and multi-cycle path and false path constraint propagation

through the floorplan hierarchy. The goal is to identify fundamental timing closure risks at floorplan stage before committing to detailed placement and routing.

---

#### **Q4. What documents/reports are generated as outputs of floorplan verification? (Cadence, Synopsys)**

*Asked at: Cadence, Synopsys, Mentor*

Floorplan verification generates: (1) Legality report (list of all violations with coordinates and severity). (2) Congestion report (overflow statistics, heat map, worst congested tile details). (3) Utilization report (global and per-region utilization percentages). (4) IR drop heat map and summary report (worst IR drop location and magnitude). (5) Timing estimation report (worst path slack and critical path summary). (6) Floorplan DEF file (the verified floorplan in standard format for handoff to placement). (7) Constraint files (updated SDC, UPF reflecting floorplan decisions). (8) Signoff checklist documenting pass/fail status of all verification criteria with engineer sign-off.

---

## **19. Power Planning**

---

#### **Q1. What is power planning in floorplanning and what structures does it create? (Intel, NVIDIA)**

*Asked at: Intel, NVIDIA, Qualcomm*

Power planning creates the physical power distribution network (PDN) that delivers VDD and VSS to all cells across the chip. Structures created include: (1) Power rings — VDD/VSS metal rings encircling the core on upper metal layers for low-resistance supply. (2) Power stripes — wide metal strips running horizontally and vertically across the core at regular pitch, forming a grid. (3) Standard cell rails — horizontal M1 power rails in each placement row connecting to stripes via vias. (4) Macro power connections — dedicated stripes aligned to macro VDD/VSS pins. (5) Via stacks — vertical connections between power structures on different metal layers.

---

#### **Q2. How is power stripe width and pitch determined? (STMicroelectronics, AMD)**

*Asked at: STMicroelectronics, AMD, Google*

Power stripe width and pitch are determined by IR drop analysis. Required: (1) Estimate total current (from power analysis — dynamic + leakage) and its distribution across the die. (2) Set target maximum IR drop (typically less than 5% of VDD). (3) Calculate required metal cross-sectional area for that current using electromigration and resistance limits:  $\text{Resistance} = R_{\text{sheet}} * \text{Length} / \text{Width}$ . (4) Determine stripe width to carry required current within EM limits. (5) Set pitch to ensure every cell is within half-pitch of a stripe for adequate current delivery. Wider stripes reduce IR drop but consume routing tracks; tighter pitch improves uniformity but uses more tracks. Iteration with IR drop tools finalizes the design.

---

#### **Q3. What is electromigration (EM) and how does it affect power planning? (Apple, Broadcom)**

*Asked at: Apple, Broadcom, TSMC*

Electromigration is the gradual physical movement of metal atoms caused by momentum transfer from electrons flowing through conductors (the 'electron wind'). EM causes metal voids (open circuits) and hillocks (shorts) over time, degrading reliability. Each metal layer has a maximum allowable current density (mA/micrometer width) specified by the foundry. Power stripes must be sized so their current density stays below EM limits for the required product lifetime (typically 10+ years). Wider stripes carry more current safely. EM analysis during power planning ensures every metal segment carrying supply current is adequately sized, preventing reliability failures in production silicon.

---

**Q4. What is the difference between static IR drop and dynamic IR drop? (Qualcomm, Samsung)**

*Asked at: Qualcomm, Samsung, TSMC*

Static IR drop assumes DC average current flow through the power grid resistance, representing steady-state voltage distribution. Static IR drop =  $I_{avg} * R_{grid}$ . It identifies regions with insufficient metal conductance for average power delivery. Dynamic IR drop occurs during simultaneous switching events when many cells switch simultaneously, causing instantaneous current spikes exceeding average current by 3-10x. The power grid's inductance ( $L * di/dt$ ) combined with resistance creates larger instantaneous voltage droops. Dynamic IR drop is typically more severe and harder to fix, requiring local decoupling capacitance (decap cells) to supply instantaneous current. Both analyses are required for complete power integrity verification.

---